



Polymarket: Deposit Wallet Passkey Upgrade

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

June 26, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	ERC-1271 signature checks can be gas-griefed by returnbombing signer contracts . .	4
3.2	Low Risk	4
3.2.1	Reject no-op and self-referential ownership handovers	4
3.2.2	Replace session-envelope booleans with an enum	5
3.2.3	Return zeroed signer fields on invalid session envelopes	5
3.2.4	PasskeySignerLib does not bind assertions to an approved rpIdHash context . .	6
3.2.5	WebAuthn challenge validation can be redirected to nested JSON fields	7
3.2.6	Session signer envelopes allow non-canonical ABI padding	8
3.2.7	Nested session signer authority can bypass execute session-signer restrictions . .	8
3.2.8	Out-of-range WebAuthn typeIndex can redirect the type check	9
3.2.9	Unbounded WebAuthn payload lengths can grief relayers	10
3.2.10	Legacy session signers can pre-pin migrated wallets to stale beacon implementations	10
3.3	Gas Optimization	12
3.3.1	Cache paused() in onlyPaused	12
3.3.2	Optimize the secp256k1 signer-id range check	13
3.4	Informational	14
3.4.1	LEGACY_IMPL is not strictly required to deploy legacy ERC1967 UUPS wallets	14
3.4.2	DepositWallet Polymarket V2 orders require ERC-7739-wrapped signatures	14
3.4.3	Legacy wallet tooling must stop using predictWalletAddress(address,bytes32) after Phase D	15
3.4.4	Ownership handover can inherit unsafe EIP-7702 signer policy changes	15

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jun 15th to Jun 19th the Cantina team conducted a review of `deposit-wallet` on commit hash `c08ba3ce`. The team identified a total of **17** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	10	3	7
Gas Optimizations	2	1	1
Informational	4	0	4
Total	17	5	12

2.1 Scope

The security review had the following components in scope for `deposit-wallet` on commit hash `c08ba3ce`:

```
src/  
├── DepositWalletFactory.sol  
├── DepositWallet.sol  
├── Errors.sol  
├── Events.sol  
├── interfaces/  
│   └── IDepositWallet.sol  
├── libraries/  
│   ├── Ownable.sol  
│   ├── PasskeySignerLib.sol  
│   ├── SessionSignerLib.sol  
│   ├── SignatureVerifierLib.sol  
│   └── WebAuthnNative.sol
```

3 Findings

3.1 Medium Risk

3.1.1 ERC-1271 signature checks can be gas-griefed by returnbombing signer contracts

Severity: Medium Risk

Context: [SignatureVerifierLib.sol#L56](#)

Description: The ERC-1271 return value check present in `SignatureVerifierLib.isValidSignatureNow()` requires `result.length == 0x20` before accepting a contract signature preventing a signer from returning oversized data that is later decoded as valid. However, this does not fully prevent `returndata` bombing because the `returndata` is fully copied into memory before the length check is executed.

The low-level `staticcall` result is returned as a dynamic `bytes` value. A malicious or misconfigured ERC-1271 signer can return very large `returndata`, forcing the caller to pay memory expansion and copy costs before the wallet checks that `result.length == 0x20`. The oversized response will ultimately fail validation, but it can cause unexpected out-of-gas instead of a clean `false / InvalidSignature` path.

While it does not enable signature forgery, magic-value bypass, or unauthorized execution, this is relevant for any path where the wallet accepts a contract signer, including owner validation, session signer validation, nested ERC-1271 verification, and ownership handover. A malicious or misconfigured contract signer can return oversized `returndata` from `isValidSignature()`, causing the relayer/operator that pays for user execution to waste gas or hit an out-of-gas failure before the signature check cleanly rejects. This can fail `DepositWallet` batch execution or integration with Polymarket V2, where the Exchange validates the maker's ERC-1271 signature during order matching.

Recommendation: Avoid copying unbounded ERC-1271 `returndata` into memory before checking its size. Use an assembly `staticcall` that caps copied `returndata` to 32 bytes, then separately checks `returndatasize() == 0x20` and compares the returned word against the expected ERC-1271 magic value.

For example, the verifier should only copy the first word of `returndata` and reject unless the actual `returndata` size is exactly 32 bytes. This preserves the strict full-word magic-value check while removing the `returndata` bomb gas griefing surface.

Polymarket: Fixed in [PR 71](#).

Cantina Managed: Verified. `Returndata` copying is now capped at one word.

3.2 Low Risk

3.2.1 Reject no-op and self-referential ownership handovers

Severity: Low Risk

Context: [Ownable.sol#L161-L170](#)

Description: The internal function `_transferOwnership` in `src/libraries/Ownable.sol` initiates a two-step ownership transfer. The function only validates that `_newOwner` is not `address(0)`, allowing two problematic targets:

1. **The current owner (`owner()`):** This creates a meaningless no-op handover that consumes gas, increments the pending owner nonce, and emits a redundant `OwnershipHandoverRequested` event.
2. **The contract itself (`address(this)`):** The contract can never finalize this handover because `SignatureVerifierLib.isValidSignatureNow` (defined in `src/libraries/SignatureVerifierLib.sol`) explicitly returns `false` if the proposed signer

matches `address(this)` . This leaves the wallet in a pending handover state that cannot be resolved through the normal signing path, requiring the owner to manually cancel the handover.

Recommendation: Prevent no-op and dead-end ownership handovers by checking that `_newOwner` is neither the current owner nor the contract itself inside `_transferOwnership` :

If more specific diagnostic messaging is preferred, new custom errors (e.g. `CannotTransferToSelf()`) can be defined.

Polymarket: Fixed in PR 71.

Cantina Managed: Verified. Ownership transfer now rejects these cases.

3.2.2 Replace session-envelope booleans with an enum

Severity: Low Risk

Context: `SessionSignerLib.sol#L29-L33`

Description: The helper function `decodeSessionSigner` in `src/libraries/SessionSignerLib.sol` returns two boolean flags `hasMagic` and `valid` to represent the parsed state of a signature envelope:

```
function decodeSessionSigner(bytes calldata _signature)
    internal
    pure
    returns (bool hasMagic, bool valid, bytes32 signerId, uint256 signerKind)
```

This design models a three-state space using two booleans:

- `hasMagic = false, valid = false` : The signature is not a session signer envelope.
- `hasMagic = true, valid = false` : The signature is a session envelope but is malformed.
- `hasMagic = true, valid = true` : The signature is a well-formed session envelope.

The combination `hasMagic = false, valid = true` is invalid and impossible. Relying on two booleans rather than a dedicated type decreases code readability, makes the API easier to misinterpret, and increases complexity if new envelope states are added.

Recommendation: Replace the boolean pair with a custom enum to model the envelope state explicitly in the type system:

```
enum SessionEnvelopeStatus {
    NotSessionEnvelope,
    MalformedEnvelope,
    WellFormedEnvelope
}
```

Update `decodeSessionSigner` to return this enum alongside the parsed fields, and adapt callers (such as `isSessionSignerSignature`) to branch on the enum values instead.

Polymarket: Fixed in PR 71.

Cantina Managed: Verified. An enum is now returned alongside the parsed fields.

3.2.3 Return zeroed signer fields on invalid session envelopes

Severity: Low Risk

Context: `SessionSignerLib.sol#L55-L65`

Description: The helper function `decodeSessionSigner` in `src/libraries/SessionSignerLib.sol` extracts session signer fields from signature payloads. If an envelope has the session signer magic suffix but fails structural layout validation, the function returns the parsed `signerId` and `signerKind` along with a `valid = false` flag:

```
if (dynamicOffset != 0x60 || innerLength > type(uint256).max - 0x1f) {
    return (true, false, signerId, signerKind);
}

uint256 paddedInnerLength = (innerLength + 0x1f) & ~uint256(0x1f);
if (paddedInnerLength > type(uint256).max - 0xa0) {
    return (true, false, signerId, signerKind);
}
if (_signature.length != 0xa0 + paddedInnerLength) {
    return (true, false, signerId, signerKind);
}
```

Since `signerId` and `signerKind` are retrieved directly from calldata:

```
assembly ("memory-safe") {
    signerId := calldataload(_signature.offset)
    signerKind := calldataload(add(_signature.offset, 0x20))
    //...
}
```

returning these fields on malformed envelope paths propagates unvalidated, user-controlled inputs. While current callers check the `valid` flag, returning attacker-controlled data on failure branches reduces the API's robustness and exposes future integrations to potential bugs if the validity check is omitted or bypassed.

Recommendation: Return zero values (`bytes32(0)` and `0`) for the `signerId` and `signerKind` fields on all invalid envelope paths:

```
if (dynamicOffset != 0x60 || innerLength > type(uint256).max - 0x1f) {
    return (true, false, bytes32(0), 0);
}

uint256 paddedInnerLength = (innerLength + 0x1f) & ~uint256(0x1f);
if (paddedInnerLength > type(uint256).max - 0xa0) {
    return (true, false, bytes32(0), 0);
}
if (_signature.length != 0xa0 + paddedInnerLength) {
    return (true, false, bytes32(0), 0);
}
```

Polymarket: Fixed in PR 71.

Cantina Managed: Verified. Malformed envelopes no longer leak caller-supplied values.

3.2.4 `PasskeySignerLib` does not bind assertions to an approved `rpIdHash` context

Severity: Low Risk

Context: `PasskeySignerLib.sol`#L28-L29

Description: `PasskeySignerLib` currently allows any arbitrary WebAuthn relying party context because it does not validate the `rpIdHash` in `authenticatorData`.

The P-256 signature is still cryptographically verified against the authorized passkey public key and the wallet batch digest, so this does not allow signature forgery. However, the wallet does not enforce that the assertion was produced for an approved frontend/domain/integrator. Any WebAuthn assertion with a

valid challenge and signature can be accepted regardless of the first 32 bytes of `authenticatorData`, so this weak binding makes malicious or unsupported signing surfaces harder to distinguish onchain.

Recommendation: Consider maintaining an allowlist of approved frontend/domain/integrator `rpIdHash` values, and requiring the first 32 bytes of `authenticatorData` to match one of them before accepting the P-256 signature.

Polymarket: Acknowledged. Intentional to allow cross application support.

Cantina Managed: Acknowledged.

3.2.5 WebAuthn challenge validation can be redirected to nested JSON fields

Severity: Low Risk

Context: [WebAuthnNative.sol#L276-L283](#)

Description: Currently, WebAuthn challenge validation is index-based rather than JSON-semantic. The P-256 signature still covers the exact `clientDataJSON`, so an attacker cannot modify the JSON after signing or forge a passkey signature. However, the signature submitter controls `challengeIndex` and `typeIndex`, which determine where the contract looks for `"challenge": "..."` and `"type": "webauthn.get"` inside the signed JSON bytes.

This allows a signed assertion to be interpreted differently by normal JSON parsers and by the contract if an assertion is produced from adversarial or non-canonical `clientDataJSON` containing duplicate or nested fields. For example, an SDK, wallet, or third-party verifier may inspect the top-level challenge and consider the assertion benign, while the contract can be given indexes that point to a nested challenge containing the real batch digest.

Consider the following example:

```
{
  "type": "webauthn.get",
  "challenge": "benign-or-decoy",
  "extra": {
    "type": "webauthn.get",
    "challenge": "attacker-chosen-batch-digest"
  }
}
```

A third-party wallet/SDK might parse the top-level challenge without any issue but then the contract could be given `challengeIndex` pointing at the nested field, so it accepts the signature for the malicious batch.

Recommendation: Require WebAuthn assertions to use canonical browser-produced `clientDataJSON` and reject ambiguous encodings onchain. At minimum, validate that `typeIndex` and `challengeIndex` point to the expected top-level `"type"` and `"challenge"` members, and reject duplicate or nested occurrences that could cause off-chain parsers and the contract to interpret different fields.

For third-party signing support, prefer deriving the challenge position from a canonical parser/encoder or from a strictly specified `clientDataJSON` template, rather than accepting arbitrary caller-supplied indexes into arbitrary JSON bytes.

Polymarket: Acknowledged. Classified as OZ vendor side issue.

Cantina Managed: Acknowledged.

3.2.6 Session signer envelopes allow non-canonical ABI padding

Severity: Low Risk

Context: `SessionSignerLib.sol#L59-L62`

Description: Session signer envelopes are currently ABI-decoded without enforcing that padding bytes after the inner dynamic `signature` field are zero. As a result, multiple distinct outer envelope byte strings can decode to the same signer and inner signature by modifying otherwise ignored non-canonical padding bytes.

This makes the outer envelope encoding malleable: two calldata signatures can be bitwise different while representing the same logical authorization. However, this does not appear to enable replay of an already executed batch or authorization of a different batch, because the inner signature is still verified over the wallet's batch digest and on-chain replay protection is provided by the wallet nonce.

The practical impact is non-canonical signature representation and possible off-chain ambiguity for systems that hash, cache, deduplicate, index, or compare raw signature bytes instead of the decoded signer and signed digest.

Recommendation: Consider enforcing canonical ABI encoding for session signer envelopes by requiring unused padding bytes after dynamic fields to be zero and rejecting trailing or non-canonical data. This would make each logical session signer authorization have a single accepted byte representation and avoid off-chain ambiguity.

Polymarket: Acknowledged. Not considered a security issue.

Cantina Managed: Acknowledged.

3.2.7 Nested session signer authority can bypass `execute` session-signer restrictions

Severity: Low Risk

Context: `DepositWallet.sol#L283-L302`

Description: `DepositWallet.execute()` validates a batch signature first, then decides whether to apply session-signer restrictions by inspecting only the outermost signature bytes. If `isSessionSigner` is true, `execute()` will block calls to the wallet itself and to the wallet beacon.

However, `isValidSignature()` can validate through recursive ERC-1271 contract ownership. When a wallet's owner is another wallet, Solady's nested ERC-7739 path can strip the outer wrapper and forward the inner signature to the owner wallet. This means the outer signature submitted to wallet `A` can look like an ordinary contract-owner signature, while the effective signer at a deeper level is a session signer authorized on ancestor wallet `C`.

For example:

```
A.owner() == B
B.owner() == C
C has authorized session signer S
```

`S` can sign a batch for `A` through nested ERC-1271 validation:

```
A.isValidSignature(...)
-> B.isValidSignature(...)
-> C.isValidSignature(...)
-> session signer S
```

The signature is accepted, but `A.execute()` computes `isSessionSigner` from the original outer bytes, not from the effective signer discovered during validation. If the outer wrapper is not itself a session-signer envelope, `A.execute()` treats the batch as non-session authority and skips the session-signer guards.

Impact: This lets a nested session signer perform calls that a directly visible session signer cannot perform:

- Call `A.authorizeSessionSigner(...)` ;
- Call `A.authorizePasskeySessionSigner(...)` ;
- Call `DepositWalletBeacon.optOut()` .

The direct wallet impact is bounded by the ownership topology: if `A` intentionally made `B` its owner, and that owner chain ultimately delegates to `S` , then `A` has accepted meaningful risk from that delegation. The protocol-relevant issue is the classification mismatch. A lower-trust session signer can route authority through recursive wallet ownership so that top-level execution policy, relayer policy, monitoring, or attribution sees an owner-style signature rather than session-signer authority.

This already existed to some extent in the previous version of the wallet. A user Alice could set themselves as a session signer and obfuscate their intentions from the operator who submits their transactions. But the mechanism here is novel and so we report it even though the risks of bypassing intended behavior are already mitigated by the operator themselves with the threat of banning anyone who performs unwanted behavior (e.g. ghost fills).

Recommendation: Do not enforce session-signer restrictions from the outer signature bytes alone. The execution path should enforce restrictions based on the effective authority used during validation.

For example, `isValidSignature()` or a dedicated `execute()` validation helper could return whether any nested layer validated through a session signer, and `execute()` should apply the self-call and beacon-call restrictions whenever the effective authorization path includes session authority.

Alternatively, require direct owner authority for sensitive targets such as `address(this)` and the wallet beacon, or reject nested session-signer envelopes when validating an `execute()` batch that would otherwise be classified as owner-authorized.

Polymarket: Acknowledged. It should not be trivial for users to be able to change ownership of their `DepositWallet` . If they do manage to achieve it, they assume all the risks with it.

Cantina Managed: Acknowledged.

3.2.8 Out-of-range WebAuthn `typeIndex` can redirect the type check

Severity: Low Risk

Context: `WebAuthnNative.sol#L148-L149`

Description: `WebAuthnNative._validateExpectedTypeHash()` validates the `WebAuthn "type": "webauthn.get"` field using a caller-supplied `typeIndex` . The function performs its bounds check and memory read in assembly, but it does not reject out-of-range indexes before computing `typeIndex + 20` .

Because the addition is unchecked, a very large `typeIndex` can wrap around and satisfy the bounds check even though it is not a valid offset into `clientDataJSON` . The subsequent equality check may then read from another memory location instead of from `clientDataJSON[typeIndex]` .

This does not allow an attacker to forge a passkey signature or modify the signed `clientDataJSON` : the P-256 signature still covers the exact `clientDataJSON` bytes. However, the `"type": "webauthn.get"` validation can be made to pass even when the actual signed JSON does not contain that string at the claimed index. Depending on the resulting memory read, malformed inputs may also trigger unexpected out-of-gas behavior, creating a grieving vector against the relayer/operator paying for verification.

Recommendation: Reject out-of-range `typeIndex` values before entering the assembly check. For example, require that the expected type string fits entirely within `clientDataJSON` before loading from memory:

```
if (typeIndex > bytes(clientDataJSON).length - 21) return false;
```

Alternatively, perform the check in overflow-safe Solidity or use saturating addition before the memory read. This ensures the verifier only reads from the intended `clientDataJSON` bytes and cleanly rejects malformed indices.

Polymarket: Acknowledged. Classified as OZ vendor side issue.

Cantina Managed: Acknowledged.

3.2.9 Unbounded WebAuthn payload lengths can grief relayers

Severity: Low Risk

Context: [WebAuthnNative.sol#L120-L125](#), [WebAuthnNative.sol#L296-L297](#)

Description: `WebAuthnNative.tryDecodeAuth` checks that the dynamic `authenticatorData` and `clientDataJSON` lengths fit inside the supplied calldata but it does not impose any maximum size on either field.

As a result, a caller can submit very large but well-formed WebAuthn payloads. If the payload passes the decoder's bounds checks, verification later hashes and copies the full `authenticatorData` and `clientDataJSON` values as part of the WebAuthn signature check. The signature will still fail unless it is valid for the authorized passkey and challenge, so this does not enable signature forgery or unauthorized execution. However, it can force the relayer/operator paying for execution to spend excessive gas, or cause verification to run out of gas before cleanly rejecting.

Recommendation: Enforce reasonable maximum lengths for `authenticatorData` and `clientDataJSON` before verification. The caps should be large enough to support expected browser-generated WebAuthn assertions, but small enough to prevent calldata-sized payloads from being hashed/copied during signature validation.

Polymarket: Acknowledged. Classified as OZ vendor side issue.

Cantina Managed: Acknowledged.

3.2.10 Legacy session signers can pre-pin migrated wallets to stale beacon implementations

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: Legacy `DepositWallet` session signers existed before the beacon opt-out mechanism was introduced. In v1.0.0, session-signed `execute()` batches only prevented calls to the wallet itself; they did not prevent calls to the future `DepositWalletBeacon`. The beacon opt-out feature later added an explicit session-signer restriction against calling the beacon, which indicates this target was intended to be owner-controlled once beacon pinning existed.

During the migration window, a currently authorized legacy session signer can submit a normal V1 session-signed batch that calls `DepositWalletBeacon.optOut()` from the wallet. The beacon writes `pinnedImplementation[msg.sender]`, so the entry is keyed to the wallet address. When that legacy wallet later migrates through `BeaconForwarder`, the forwarder resolves the active implementation through `BEACON.implementation()` from the wallet execution context, causing the beacon to see `msg.sender == wallet` and return the pre-pinned implementation.

As a result, a lower-trust legacy session signer can create an owner-unapproved upgrade opt-out that survives migration. The wallet will continue using the stale pinned implementation even after the beacon default is upgraded, so future beacon-delivered fixes or policy changes will not reach that wallet unless the owner explicitly opts back in. This is most impactful for wallets that are trusted as ERC-1271 signers by other wallets or external integrations, because downstream consumers may continue trusting a wallet that silently remains on a stale implementation.

Proof of Context:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.15;

import {WalletHelpers} from "@deposit-wallet/src/libraries/WalletHelpers.sol";
import {Batch, Call, WalletLib} from "@deposit-wallet/src/libraries/WalletLib.sol";
import {SessionSignerLib} from "@deposit-wallet/src/libraries/SessionSignerLib.sol";
import {DepositWalletBeacon} from "@deposit-wallet/src/DepositWalletBeacon.sol";
import {
    BeaconForwarderMigrationTests,
    LegacyWalletFixture,
    LegacyV1SessionWalletFixture
} from "../BeaconForwarderMigration.t.sol";

/// @notice Marker implementation used to confirm non-pinned wallets follow
///     default-beacon updates.
contract MarkerWalletImpl {
    function marker() external pure returns (uint256) {
        return 0xDEADBEEF;
    }

    function walletInterfaceId() external pure returns (bytes32) {
        return keccak256("Polymarket.DepositWallet");
    }

    fallback() external payable {}
    receive() external payable {}
}

/// @notice Reuses legacy migration harness setup from `BeaconForwarderMigrationTests`.
contract LegacySessionPrePinTests is BeaconForwarderMigrationTests {
    using WalletHelpers for uint256;
    using WalletLib for Batch;

    function _legacyImplementation() internal override returns (address) {
        return address(new LegacyV1SessionWalletFixture());
    }

    function _deployLegacySessionWallet(bytes32 _id)
        internal
        returns (LegacyV1SessionWalletFixture wallet)
    {
        wallet =
            LegacyV1SessionWalletFixture(payable(address(_deployLegacyWallet(_id))));
    }

    function test_legacy_session_signer_can_pre_pin_before_migration_and_survive_forward_upgrade()
        public {
        LegacyV1SessionWalletFixture pinnedWallet =
            _deployLegacySessionWallet(bytes32(uint256(1)));
        LegacyV1SessionWalletFixture unpinnedWallet =
            _deployLegacySessionWallet(bytes32(uint256(2)));

        pinnedWallet.setOwner(alice);
        unpinnedWallet.setOwner(alice);

        uint256 sessionKey = vm.createWallet("legacy-session-signer").privateKey;
    }
}
```

```

    address sessionSigner = vm.addr(sessionKey);

    pinnedWallet.setSessionSigner(sessionSigner, block.timestamp + 1 days);

    Call memory optOut = Call({
        target: address(beacon),
        value: 0,
        data: abi.encodeWithSelector(DepositWalletBeacon.optOut.selector)
    });

    Call[] memory calls = new Call[](1);
    calls[0] = optOut;

    Batch[] memory batches = new Batch[](1);
    batches[0] =
        Batch({wallet: address(pinnedWallet), nonce: 0, calls: calls, deadline:
            ↪ block.timestamp + 1 minutes});

    bytes[] memory signatures = new bytes[](1);
    signatures[0] = SessionSignerLib.getSessionSignerSignature(
        sessionSigner,
        sessionKey.sign(batches[0], address(pinnedWallet))
    );

    vm.prank(operator);
    walletFactory.proxy(batches, signatures);

    assertEq(beacon.pinnedImplementation(address(pinnedWallet)),
        ↪ walletImplementation);

    _migrateToBeaconForwarder(pinnedWallet);
    _migrateToBeaconForwarder(unpinnedWallet);

    MarkerWalletImpl marker = new MarkerWalletImpl();
    vm.prank(owner);
    beacon.upgradeTo(address(marker));

    vm.prank(address(pinnedWallet));
    assertEq(beacon.implementation(), walletImplementation);

    vm.prank(address(unpinnedWallet));
    assertEq(beacon.implementation(), address(marker));
}
}

```

Recommendation: Add migration preflight checks that reject or explicitly flag any wallet with pre-existing beacon pins where `DepositWalletBeacon.pinnedImplementation(wallet) != address(0)`. Monitor `OptedOut` and `OptedIn` events during the migration window, especially for legacy wallets that have not yet migrated. The operator should also refuse V1 session-signed batches that target the beacon.

Polymarket: Acknowledged. We will ban these wallets from using our trading engine offchain.

Cantina Managed: Acknowledged.

3.3 Gas Optimization

3.3.1 Cache `paused()` in `onlyPaused`

Severity: Gas Optimization

Context: `DepositWallet.sol#L113-L120`

Description: The `onlyPaused` modifier in `src/DepositWallet.sol` reads the paused state twice:

```
modifier onlyPaused() {
    require(paused() != 0, NotPaused());
    require(
        block.timestamp > paused() + DepositWalletFactory(factory()).timelockDelay(),
        TimelockInsufficientDelay()
    );
    _;
}
```

Because `paused()` reads the `_PAUSED_SLOT` storage slot using inline assembly (`sload`), executing it twice results in a redundant storage read, causing unnecessary gas usage.

Recommendation: Cache the returned timestamp of `paused()` in a local variable to read the storage slot only once:

```
modifier onlyPaused() {
    uint256 pausedAt = paused();
    require(pausedAt != 0, NotPaused());
    require(
        block.timestamp > pausedAt + DepositWalletFactory(factory()).timelockDelay(),
        TimelockInsufficientDelay()
    );
    _;
}
```

Polymarket: Fixed in PR 71.

Cantina Managed: Verified. The paused timestamp is now cached.

3.3.2 Optimize the secp256k1 signer-id range check

Severity: Gas Optimization

Context: `SessionSignerLib.sol#L67-L72`

Description: In `src/libraries/SessionSignerLib.sol`, the function `decodeSessionSigner` validates that a `SESSION_SIGNER_KIND_SECP256K1` signer ID represents a valid, non-zero EVM address.

Recommendation: This validation uses multiple operations (a bitwise right-shift, two comparisons, and type casting). This logic can be simplified into a single arithmetic check:

```
unchecked {
    valid = uint256(signerId) - 1 < type(uint160).max;
}
```

- **When `signerId` is 0:** `uint256(signerId) - 1` underflows to `2**256 - 1`, which is not less than `type(uint160).max`, yielding `valid = false`.
- **When `signerId` has bits set in the high 96 bits:** The subtracted value remains greater than or equal to `type(uint160).max`, yielding `valid = false`.
- **When `signerId` is a valid address in `[1, 2**160 - 1]`:** The subtracted value falls in `[0, 2**160 - 2]`, which is strictly less than `type(uint160).max`, yielding `valid = true`.

Replacing the shift and double comparisons with this single unchecked check saves gas on a critical verification hot path.

Polymarket: Acknowledged. Preferred for readability.

Cantina Managed: Acknowledged.

3.4 Informational

3.4.1 `LEGACY_IMPL` is not strictly required to deploy legacy ERC1967 UUPS wallets

Severity: Informational

Context: `DepositWalletFactory.sol#L155-L158`

Description: `DepositWalletFactory` introduces the immutable `LEGACY_IMPL` as a reference to the existing live `DepositWallet` implementation so `deployLegacy()` can reproduce exact pre-beacon ERC-1967 UUPS wallet addresses. However, this is not strictly necessary since the deprecated storage slot 0 still references the v1 implementation.

Recommendation: Given upgrade documentation explicitly states that legacy `DepositWalletFactory` storage layout should be preserved, consider repurposing the deprecated storage slot 0 instead of explicitly introducing an immutable.

Polymarket: Acknowledged. Explicit bytecode read preferred.

Cantina Managed: Acknowledged.

3.4.2 `DepositWallet` Polymarket V2 orders require ERC-7739-wrapped signatures

Severity: Informational

Context: `DepositWallet.sol#L689-L691`

Description: `DepositWallet` orders submitted through Polymarket V2 must be signed and encoded using the ERC-7739 nested signature format expected by the wallet.

Polymarket V2's generic smart-wallet path is `POLY_1271`, which validates signatures by calling `isValidSignature()` on `order.maker` and requires `order.signer == order.maker`. For `DepositWallet` orders, this means `maker == signer == depositWallet`.

The Exchange calls `DepositWallet.isValidSignature()` directly during order matching. From the wallet's perspective, the Exchange is not a safe ERC-1271 caller, because `DepositWallet` treats only `DepositWalletFactory` as safe. As a result, Solady's ERC-1271 implementation expects the submitted signature bytes to contain an ERC-7739 nested signature over the Polymarket V2 order digest. A raw owner, session-signer, or passkey signature over the Polymarket V2 order digest will fail validation even if it was produced by an otherwise authorized signer.

This is an off-chain integration requirement rather than an on-chain authorization bypass. The wallet rejects incorrectly encoded signatures, but clients, SDKs, operators, and third-party integrations must generate the wrapped signature format or valid user intent may fail during `matchOrders()`.

Recommendation: Document the required `DepositWallet` signing flow for use with Polymarket V2.

For `POLY_1271` orders, clients should set `maker == signer == depositWallet` and generate the Solady ERC-7739 nested signature payload expected by `DepositWallet.isValidSignature()`. The inner authorization may then be an owner, session-signer, or passkey signature, but it must be wrapped for the Polymarket Exchange domain rather than submitted as a raw signature over the order digest.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.4.3 Legacy wallet tooling must stop using `predictWalletAddress(address,bytes32)` after Phase D

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: After the Phase D factory upgrade, the legacy two-argument `predictWalletAddress(address,bytes32)` ABI is retained for backwards compatibility, but its semantics change. The `_implementation` argument is ignored, and the function returns the new beacon-shape deposit wallet address.

This is safe for newly issued beacon wallets, but it is not safe for tooling that still expects `predictWalletAddress(LEGACY_IMPL, id)` to return a legacy ERC-1967 UUPS deposit wallet address. For existing Polygon users who have funds or state in legacy wallets at the old deterministic address, legacy-aware tooling must resolve the legacy wallet with `predictLegacyWalletAddress(id)` instead.

If off-chain clients, recovery tools, relayer logic, or migration scripts continue calling the legacy two-argument predictor for existing users after Phase D, they may bind to the empty beacon wallet rather than the funded legacy wallet. This does not move or compromise funds on-chain, but it can cause wallet deployment and recovery to operate on the wrong wallet address. Given that wallet addresses are deterministically deployed with one-shot `CREATE3` salts, using the salt for the wrong wallet type or identifier on a given chain can permanently prevent the recovery of funds previously sent to the intended counterfactual legacy wallet address.

Recommendation: Ensure that the post-Phase-D wallet-resolution rules are explicitly documented and update all off-chain consumers. Use `predictWalletAddress(bytes32)` or `predictWalletAddress(address,bytes32)` only for the current beacon proxy wallet path. For legacy Polygon wallets or any id whose legacy address was handed out before the upgrade, use `predictLegacyWalletAddress(bytes32)` and `deployLegacy(...)` where deployment is required.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.4.4 Ownership handover can inherit unsafe EIP-7702 signer policy changes

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: `DepositWallet` ownership handover completion validates the pending owner at completion time using the pending owner's current signature policy. Since contract owners and EIP-7702 delegated EOAs are supported, this means an address that was an ordinary EOA when `transferOwnership()` was requested may later be evaluated as a code-bearing ERC-1271 signer when `completeOwnershipHandover()` is called.

This can matter if the pending owner opts into an unsafe EIP-7702 delegation before completing the handover. EIP-7702 delegation does not automatically initialize delegated account storage. If the delegated implementation exposes a public initializer, such as `initialize(owner)`, and the pending owner or their tooling does not initialize it atomically, a third party may be able to initialize the delegated account and control its ERC-1271 signature policy.

In such a scenario, the attacker could satisfy the pending owner's ERC-1271 approval and complete the victim wallet's ownership handover, obtaining effective owner-level control through the pending owner address. The attacker cannot force the pending owner to delegate, so this requires unsafe 7702 account setup or tooling by the pending owner rather than a direct wallet bypass. However, the wallet currently does not pin the pending owner's signer class at request time, so this risk should be documented for supported EIP-7702 owners.

Recommendation: Document that EIP-7702 delegated owners and pending owners must use account implementations with safe initialization semantics. Delegation setup should either be atomic with initialization, require an authorization from the EOA, or avoid public initialization entirely.

Consider pinning the pending owner's signer class at `transferOwnership()` time. For example, if the pending owner has no code when the handover is requested, require ECDSA-only acceptance from that address when completing the handover. Alternatively, reject or add explicit policy for EIP-7702 delegated pending owners unless the delegated account is already initialized and trusted.

Polymarket: Acknowledged. Transfer Ownership is an operator-gated process. We only allow this for our own trusted migration intents, users are not independently allowed to execute this.

Cantina Managed: Acknowledged.